

**FACULDADE GRAN TIETÊ**  
**ENGENHARIA DA COMPUTAÇÃO**  
VINICIUS FERNANDES ESCOBEDO

**MÉTODOS NUMÉRICOS – TRABALHO 1º BIMESTRE**

**BARRA BONITA**

**2023**

## INTRODUÇÃO

A busca por números primos é um dos problemas mais antigos e fundamentais da matemática e da ciência da computação, com implicações que vão desde a criptografia moderna até a teoria dos números. No entanto, os desafios computacionais associados a essa busca tornam-se cada vez mais complexos à medida que os números aumentam, exigindo algoritmos sofisticados e otimizações criativas. Além disso, a relação entre números primos e números perfeitos, como demonstrado pelo teorema de Euclides-Euler, revela conexões profundas que continuam a inspirar pesquisas interdisciplinares.

## DESAFIOS COMPUTACIONAIS NA BUSCA POR PRIMOS

Um dos principais obstáculos na identificação de números primos reside na complexidade algorítmica. Algoritmos ingênuos, como a divisão trial (testar divisores até  $\sqrt{n}$ ), tornam-se impraticáveis para números com centenas de dígitos, uma vez que sua complexidade é  $O(\sqrt{n})$ . Para contextos práticos, como a criptografia RSA, que utiliza primos com milhares de bits, métodos mais eficientes são essenciais. A fatoração de números grandes, problema intimamente ligado à primalidade, também é exponencialmente difícil, conforme destacado por Cormen et al. em *Introduction to Algorithms* (2009), o que fundamenta a segurança de sistemas criptográficos.

A escalabilidade é outro desafio crítico. Gerar todos os primos até um limite  $N$  exige estratégias que equilibrem tempo e memória. Por exemplo, o crivo de Eratóstenes, desenvolvido na Grécia Antiga, possui complexidade  $O(N \log \log N)$  em tempo, mas demanda  $O(n)$  em memória, tornando-o inviável para  $N > 10^{12}$ . Soluções como o *crivo segmentado*, que divide o intervalo de busca em blocos menores, foram propostas para contornar esse limite.

A busca por primos recordes, como os primos de Mersenne (da forma  $2^p - 1$ ), ilustra a interseção entre teoria e computação de alto desempenho. Projetos como o GIMPS (Great Internet Mersenne Prime Search) utilizam o teste de Lucas-Lehmer, específico para primos de Mersenne, e distribuem cálculos em milhares de computadores, demonstrando como a colaboração global e o *hardware* moderno são indispensáveis para explorar fronteiras matemáticas.

## ALGORITMOS CLÁSSICOS E MODERNOS

Entre os métodos históricos, o Crivo de Eratóstenes (século III a.C.) permanece relevante. Seu princípio — eliminar sistematicamente múltiplos de cada primo descoberto — foi aprimorado ao longo dos séculos. Implementações modernas, como as descritas por Sedgewick e Wayne em *Algorithms* (2011), ignoram múltiplos pares e utilizam estruturas de dados compactas, reduzindo o uso de memória. Apesar disso, sua aplicação direta ainda é limitada por escala, o que motivou alternativas como o Crivo de Sundaram.

Proposto pelo matemático indiano S. P. Sundaram em 1934, o Crivo de Sundaram foca em gerar números ímpares compostos através da expressão  $i + j + 2ij$ , deixando os restantes como candidatos a primos (excluindo o 2). Apesar de sua elegância teórica, sua eficiência prática é inferior à do crivo de Eratóstenes para  $N$  grandes, conforme comparado por Pomerance e Crandall em *Prime Numbers: A Computational Perspective* (2005).

Já o Crivo de Atkin, desenvolvido por A. O. L. Atkin e Daniel J. Bernstein em 2003, representa um avanço teórico ao utilizar formas quadráticas para identificar primos. Segundo os autores, sua complexidade teórica é  $O(N/\log \log N)$ , superando o crivo de Eratóstenes. No entanto, constantes ocultas e a dificuldade de implementação limitaram sua adoção prática. Sendo assim otimizações do crivo de Eratóstenes ainda são mais eficazes na maioria dos cenários reais.

## TESTE PRÁTICO DOS ALGORÍTMOS E COMPARAÇÃO DA EXECUÇÃO

Os algoritmos a seguir são projetados para encontrar números primos usando os diferentes métodos apresentados e serão executados por 20 segundos, contabilizando o máximo de primos encontrados. Cada algoritmo é disparado usando *threads* para permitir a contagem de tempo e a geração de primos simultaneamente.

## Algoritmo simples

```
1 import time
2 import sys
3 import threading
4
5 EXECUTION_TIME = 20
6 prime_count = 0
7
8 def is_prime(n):
9     if n < 2:
10         return False
11     for i in range(2, int(n ** 0.5) + 1):
12         if n % i == 0:
13             return False
14     return True
15
16 def generate_primes(stop_event):
17     global prime_count
18     num = 0
19     while not stop_event.is_set():
20         if is_prime(num):
21             prime_count += 1
22         num += 1
23
24 if __name__ == "__main__":
25     print(f"Iniciando a contagem de números primos por {EXECUTION_TIME} segundos...")
26     start_time = time.time()
27     stop_event = threading.Event()
28
29     prime_thread = threading.Thread(target=generate_primes, args=(stop_event,))
30     prime_thread.start()
31
32     try:
33         while True:
34             current_time = time.time() - start_time
35             if current_time >= EXECUTION_TIME:
36                 stop_event.set()
37                 break
38             sys.stdout.write(f"\rTempo corrente: {current_time:.2f} segundos")
39             sys.stdout.flush()
40     except KeyboardInterrupt:
41         stop_event.set()
42
43     prime_thread.join()
44     print(f"\nNúmeros primos encontrados em {int(current_time)} segundos: {prime_count}")
```

Este script usa a verificação direta de primalidade. A função `is_prime` verifica cada número sequencialmente para determinar se é primo. Se for, incrementa `prime_count`. A função `generate_primes` executa essa verificação em um loop até que o evento de parada (`stop_event`) seja acionado. A thread principal monitora o tempo e atualiza a exibição do tempo corrente. Quando o tempo atinge 20 segundos, a thread de geração de primos é interrompida.

## Algoritmo usando Crivo de Eratóstenes

```
1 import time
2 import sys
3 import threading
4
5 EXECUTION_TIME = 20
6 prime_count = 0
7
8 def sieve_of_eratosthenes(limit):
9     primes = []
10    sieve = [True] * (limit + 1)
11    for num in range(2, limit + 1):
12        if sieve[num]:
13            primes.append(num)
14            for multiple in range(num * num, limit + 1, num):
15                sieve[multiple] = False
16    return primes
17
18 def generate_primes(stop_event):
19    global prime_count
20    limit = 500000 # Limite inicial para a Sieve of Eratosthenes
21    while not stop_event.is_set():
22        primes = sieve_of_eratosthenes(limit)
23        prime_count = len(primes)
24        limit *= 2 # Aumenta o limite para encontrar mais primos
25
26 if __name__ == "__main__":
27    print(f"Iniciando a contagem de números primos por {EXECUTION_TIME} segundos usando o método de Sieve of Eratosthenes...")
28    start_time = time.time()
29    stop_event = threading.Event()
30
31    prime_thread = threading.Thread(target=generate_primes, args=(stop_event,))
32    prime_thread.start()
33
34    try:
35        while True:
36            current_time = time.time() - start_time
37            if current_time >= EXECUTION_TIME:
38                stop_event.set()
39                break
40            sys.stdout.write(f"\rTempo corrente: {current_time:.2f} segundos")
41            sys.stdout.flush()
42    except KeyboardInterrupt:
43        stop_event.set()
44
45    prime_thread.join()
46    print(f"\nNúmeros primos encontrados em {int(current_time)} segundos: {prime_count}")
```

A função `sieve_of_eratosthenes` marca múltiplos de cada número primo começando de 2, eliminando-os da lista de candidatos a primos. A função `generate_primes` executa o crivo de Eratosthenes em um loop, aumentando o limite e atualizando `prime_count` com o número de primos encontrados. A thread principal monitora o tempo e atualiza a exibição do tempo corrente. Quando o tempo atinge 20 segundos, a thread de geração de primos é interrompida.

## Algoritmo usando Crivo de Sundaram

```
1 import time
2 import sys
3 import threading
4
5 EXECUTION_TIME = 20
6 prime_count = 0
7
8 def sieve_of_sundaram(limit):
9     n = (limit - 1) // 2
10    sieve = [True] * (n + 1)
11    for i in range(1, n + 1):
12        j = i
13        while i + j + 2 * i * j <= n:
14            sieve[i + j + 2 * i * j] = False
15            j += 1
16    primes = [2] + [2 * i + 1 for i in range(1, n + 1) if sieve[i]]
17    return primes
18
19 def generate_primes(stop_event):
20     global prime_count
21     limit = 100000 # Limite inicial para a Sieve of Sundaram
22     while not stop_event.is_set():
23         primes = sieve_of_sundaram(limit)
24         prime_count = len(primes)
25         limit *= 2 # Aumenta o limite para encontrar mais primos
26
27 if __name__ == "__main__":
28     print(f"Iniciando a contagem de números primos por {EXECUTION_TIME} segundos usando o método de Sieve of Sundaram...")
29     start_time = time.time()
30     stop_event = threading.Event()
31
32     prime_thread = threading.Thread(target=generate_primes, args=(stop_event,))
33     prime_thread.start()
34
35     try:
36         while True:
37             current_time = time.time() - start_time
38             if current_time >= EXECUTION_TIME:
39                 stop_event.set()
40                 break
41             sys.stdout.write(f"\rTempo corrente: {current_time:.2f} segundos")
42             sys.stdout.flush()
43     except KeyboardInterrupt:
44         stop_event.set()
45
46     prime_thread.join()
47     print(f"\nNúmeros primos encontrados em {int(current_time)} segundos: {prime_count}")
```

A função `sieve_of_sundaram` gera números primos até um limite, eliminando números não primos de uma lista. A função `generate_primes` executa o crivo de Sundaram em um loop, aumentando o limite e atualizando `prime_count` com o número de primos encontrados. A thread principal monitora o tempo e atualiza a exibição do tempo corrente. Quando o tempo atinge 20 segundos, a thread de geração de primos é interrompida.

## Algoritmo usando Crivo de Atkin

```
1 import time
2 import sys
3 import math
4 import threading
5
6 EXECUTION_TIME = 20
7 prime_count = 0
8
9 def sieve_of_atkin(limit):
10     primes = [2, 3]
11     sieve = [False] * (limit + 1)
12     for x in range(1, int(math.sqrt(limit)) + 1):
13         for y in range(1, int(math.sqrt(limit)) + 1):
14             n = 4 * x**2 + y**2
15             if n <= limit and (n % 12 == 1 or n % 12 == 5):
16                 sieve[n] = not sieve[n]
17             n = 3 * x**2 + y**2
18             if n <= limit and n % 12 == 7:
19                 sieve[n] = not sieve[n]
20             n = 3 * x**2 - y**2
21             if x > y and n <= limit and n % 12 == 11:
22                 sieve[n] = not sieve[n]
23     for n in range(5, int(math.sqrt(limit)) + 1):
24         if sieve[n]:
25             for k in range(n**2, limit + 1, n**2):
26                 sieve[k] = False
27     primes.extend([n for n in range(5, limit + 1) if sieve[n]])
28     return primes
29
30 def generate_primes(stop_event):
31     global prime_count
32     limit = 100000 # Limite inicial para a Sieve of Atkin
33     while not stop_event.is_set():
34         primes = sieve_of_atkin(limit)
35         prime_count = len(primes)
36         limit *= 2 # Aumenta o limite para encontrar mais primos
37
38 if __name__ == "__main__":
39     print(f"Iniciando a contagem de números primos por {EXECUTION_TIME} segundos usando o método de Sieve of Atkin...")
40     start_time = time.time()
41     stop_event = threading.Event()
42
43     prime_thread = threading.Thread(target=generate_primes, args=(stop_event,))
44     prime_thread.start()
45
46     try:
47         while True:
48             current_time = time.time() - start_time
49             if current_time >= EXECUTION_TIME:
50                 stop_event.set()
51                 break
52             sys.stdout.write(f"\rTempo corrente: {current_time:.2f} segundos")
53             sys.stdout.flush()
54     except KeyboardInterrupt:
55         stop_event.set()
56
57     prime_thread.join()
58     print(f"\nNúmeros primos encontrados em {int(current_time)} segundos: {prime_count}")
```

A função `sieve_of_atkin` usa fórmulas matemáticas para marcar números primos em uma lista, aplicando correções para eliminar falsos positivos. A função `generate_primes` executa o crivo de Atkin em um loop, aumentando o limite e atualizando `prime_count` com o número de primos encontrados. A thread principal monitora o tempo e atualiza a exibição do tempo corrente. Quando o tempo atinge 20 segundos, a thread de geração de primos é interrompida.

## Execução

```
C:\dev\Faculdade\trabalho-metodo-numeric>python simple_prime_finder.py
Iniciando a contagem de números primos por 20 segundos...
Tempo corrente: 19.99 segundos
Números primos encontrados em 20 segundos: 123982

C:\dev\Faculdade\trabalho-metodo-numeric>python eratosthenes_prime_finder.py
Iniciando a contagem de números primos por 20 segundos usando o método de Sieve of Eratosthenes...
Tempo corrente: 19.99 segundos
Números primos encontrados em 20 segundos: 1973815

C:\dev\Faculdade\trabalho-metodo-numeric>python sundaram_prime_finder.py
Iniciando a contagem de números primos por 20 segundos usando o método de Sieve of Sundaram...
Tempo corrente: 19.99 segundos
Números primos encontrados em 20 segundos: 837099

C:\dev\Faculdade\trabalho-metodo-numeric>python atkin_prime_finder.py
Iniciando a contagem de números primos por 20 segundos usando o método de Sieve of Atkin...
Tempo corrente: 19.98 segundos
Números primos encontrados em 20 segundos: 837099
```

Analisando a execução dos algoritmos é perceptível a discrepância de performance entre as diferentes abordagens. Como esperado, o algoritmo simples encontrou o menor número de primos dentro do tempo estipulado. Já os algoritmos de Sundaram e Atkin tiveram o mesmo desempenho conseguindo uma eficiência aproximadamente 7 vezes maior. O algoritmo de Eratóstenes obteve o melhor resultado encontrando mais de 16 vezes a quantidade de números primos do método simples.

## NÚMEROS PERFEITOS E SUA CONEXÃO COM PRIMOS

Os números perfeitos, definidos como iguais à soma de seus divisores próprios (excluindo-os), têm uma relação histórica com os primos. O teorema de Euclides-Euler, demonstrado por Euler em 1747, estabelece que todo número perfeito par é da forma  $2^{p-1}(2^p - 1)$ , onde  $2^p - 1$  é um primo de Mersenne. Isso vincula diretamente a busca por perfeitos à descoberta de primos de Mersenne, como ilustrado por  $6 = 2^{2-1}(2^2 - 1)$  e  $28 = 2^{3-1}(2^3 - 1)$ .

A existência de números perfeitos ímpares permanece um dos problemas em aberto mais intrigantes da teoria dos números. Nenhum exemplo foi encontrado, e resultados parciais, como os de Ochem e Rao (2012), sugerem que, se existirem, devem ser maiores que  $10^{1500}$ . Essa incógnita reforça a complexidade inerente à teoria dos números, onde questões aparentemente simples exigem ferramentas computacionais e teóricas avançadas.



## CONCLUSÃO

A busca por números primos e perfeitos exemplifica a simbiose entre matemática pura e computação. Algoritmos como os crivos de Eratóstenes e Atkin refletem séculos de evolução no pensamento algorítmico, enquanto projetos colaborativos como o GIMPS ilustram o poder da computação distribuída. A persistência de problemas não resolvidos, como a existência de perfeitos ímpares, mantém a área vibrante e desafiadora. Como observou Donald Knuth em *The Art of Computer Programming* (1997), a eficiência na manipulação de primos não é apenas uma questão técnica, mas uma ponte entre a abstração matemática e as limitações do mundo real.

## REFERÊNCIAS

CORMEN, Thomas H. et al. **Introduction to algorithms**. MIT press, 2022.

SEEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. Addison-wesley professional, 2011.

CRANDALL, Richard E.; POMERANCE, Carl. **Prime numbers: a computational perspective**. New York: Springer, 2005.

OCHEM, Pascal; RAO, Michaël. Odd perfect numbers are greater than  $10^{1500}$ . **Mathematics of Computation**, v. 81, n. 279, p. 1869-1877, 2012.

KNUTH, Donald Ervin. **The art of computer programming**. Pearson Education, 1997.